❶ Required ActorSystem to run your Stream, shown in the previous listing

❷ Thread pool used to run the callback attached to the Future

❸ Result of parsing, shown in the previous listing

❹ Callback to execute when the Future completes with Success to assert your expectations

To execute the future, you need the usual `ExecutionContext`. Then you can pattern-match on the `Future` callback `Success`.

To save work when testing futures, you can use `ScalaFutures` from the `ScalaTest` library to extend the `*Spec`. This way, you can skip the verbose `onComplete` pattern-matching with `Success` and the `ExecutionContext`.

Listing 14.7 Shorter assert

```
import org.scalatest.concurrent.ScalaFutures

class CSVSpec
    extends AnyFlatSpec
    with Matchers
    with ScalaFutures                          ❶
    with BeforeAndAfterAll {

  implicit val system =
    ActorSystem(Behaviors.empty, "alpakkaExamples")

  "A CSV" should "be readable in shorter test" in {

    val result: Future[Seq[List[ByteString]]] = ...

    val lines = result.futureValue             ❷
    lines.head should be(
      List(
        ByteString("Alice"),
        ByteString("Pleasence"),
        ByteString("19")))

  }
}
```

❶ Adds ScalaFutures